Indexing

Indexes

- In MongoDB, an index is a data structure that captures a subset of a collection's data in an easy to traverse form. MongoDB indexes use a B-tree data structure.
- Indexes store the value of a specific field or fields, and attach a reference to the original document to each index entry.
- Indexes support efficient execution of queries in MongoDB.
- Without indexes, MongoDB must scan every document in a collection to select those documents that match query statement.
- If an appropriate index exists for a query, MongoDB uses the index to limit the number of documents it must scan.
- The index stores the value of a specific field or set of fields, ordered by the value of the field.

Use Cases

- If your application is repeatedly running queries on the same fields, you can create an index on those fields to improve performance.
- For example:

  Scenario1: A human resources department often needs to look up employees by employee ID. You can create an index on the employee ID field to improve query performance……Single Field Index
- **Scenario2:** A salesperson often needs to look up client information by location. Location is stored in an embedded object with fields like state, city, and zipcode. You can create an index on the location object to improve performance for queries on that object……Single Field Index on an embedded document

❖ When you create an index on an embedded document, only queries that specify the entire embedded document use the index. Queries on a specific

- **Scenario3:** A grocery store manager often needs to look up inventory items by name and quantity to determine which items are low stock. You can create a single index on both the item and quantity fields to improve query performance…….. Compound Index

Default Index:

- MongoDB creates a unique index on the _id field during the creation of a collection.
- The _id index prevents clients from inserting two documents with the same value for the _id field. We cannot drop this index.

Create an Index

```
db.collection.createIndex( <key and index type specification>, <options> )
```

- A document that contains the field and value pairs where the field is the index key and the value describes the type of index for that field.
- For an ascending index on a field, specify a value of 1. For descending index, specify a value of -1.

```
db.collection.createIndex( { name: -1 } )
```

To confirm that the index was created:

- **db.collection.getIndexes()**

```
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: -1 }, name: 'name_-1' }
]
```

Specify an Index Name

- When you create an index, you can give the index a custom name.
- Giving your index a name helps distinguish different indexes on your collection.
- For example, you can more easily identify the indexes used by a query in the query plan's ***explain*** results if your indexes have distinct names.

db.collection.find(<queryFilter>).explain("executionStats")

- To specify the index name, include the name option when you create the index:

```
db.<collection>.createIndex(
    { <field>: <value> },
    { name: "<indexName>" }
)
```

Important points:

- Index names must be unique.
- Creating an index with the name of an existing index returns an error.
- You can't rename an existing index. Instead, you must drop and recreate the index with a new name.

```
test> db.records.find({age:{$gt:22}}).explain("executionStats")
{
  explainVersion: '2',
  queryPlanner: {
    namespace: 'test.records',
    indexFilterSet: false,
    parsedQuery: { age: { '$gt': 22 } },
    queryHash: 'B0DFDEAC',
    planCacheKey: '24C011CB',
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    winningPlan: {
      queryPlan: {
        stage: 'FETCH',
        planNodeId: 2,
        inputStage: {
          stage: 'IXSCAN',
          planNodeId: 1,
          keyPattern: { age: 1 },
          indexName: 'age_1',
          isMultiKey: false,
          multiKeyPaths: { age: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: { age: [ '(22, inf.0]' ] }
        }
      },
      slotBasedPlan: {
        slots: '$$RESULT=s11 env: { s5 = KS(2B2CFE04), s2 = Nothing (SEARCH_META), s10 = {"age" : 1}, s1 = TimeZoneDatabase(America/Argentina/Ushuaia...Atlantic/Azores) (ti
KS(33FFFFFFFFFFFFFFFFFE04), s3 = 1771416882432 (NOW) }',
        stages: '[2] nlj inner [] [s4, s7, s8, s9, s10] \n' +
          '    left \n' +
          '        [1] cfilter {(exists(s5) && exists(s6))} \n' +
          '        [1] ixseek s5 s6 s9 s4 s7 s8 [] @"71c42a80-52c1-4a29-8f1f-b058a15086b1" @"age_1" true \n' +
          '    right \n' +
          '        [2] limit 1 \n' +
          '        [2] seek s4 s11 s12 s7 s8 s9 s10 [] @"71c42a80-52c1-4a29-8f1f-b058a15086b1" true false \n'
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 12,
    totalKeysExamined: 2,
```

Default Index Names

- If you don't specify a name during index creation, the system generates the name by concatenating each index key field and value with underscores. For example:

Index	Default Name
`{ score : 1 }`	`score_1`
`{ content : "text", "description.tags": "text" }`	`content_text_description.tags_text`
`{ category : 1, locale : "2dsphere"}`	`category_1_locale_2dsphere`
`{ "fieldA" : 1, "fieldB" : "hashed", "fieldC" : -1 }`	`fieldA_1_fieldB_hashed_fieldC_-1`

Drop an Index

- You can remove a specific index from a collection.
- You may need to drop an index if you see a negative performance impact, want to replace it with a new index, or no longer need the index.
- To drop an index, use one of the following shell methods:

Method	Description
`db.collection.dropIndex()`	Drops a specific index from the collection.
`db.collection.dropIndexes()`	Drops all removable indexes from the collection or an array of indexes, if specified.

the _id field. To drop the _id index, you must drop the entire collection.

- If you drop an index that's actively used in production, you may experience performance degradation. Before you drop an index, consider **hiding the index** to evaluate the potential impact of the drop.

db.collection.hideIndex({age:1})

db.collection.unhideIndex({age:1})

Procedure:

- To drop an index, you need its name. To get all index names for a collection, run the getIndexes() m `db.<collection>.getIndexes()`
- After you identify which indexes to drop, use one of the following drop methods for the specified collection:

✓ **Drop a Single Index:** use the dropIndex() method and specify the index name:

```
db.<collection>.dropIndex("<indexName>")
```

✓ **Drop Multiple** `db.<collection>.dropIndexes( [ "<index1>", "<index2>", "<index3>" ] )` an array of index names:

✓ **Drop All Indexes Excep** `db.<collection>.dropIndexes()`

Results: After you d returns information about the status of the ope

```
...
{ "nIndexesWas" : 3, "ok" : 1 }
...
```

- The value of nIndexesWas reflects the number of indexes before removing an index.
- To confirm that the index was dropped, run:db.collection.getIndexes() method.
- The dropped index no longer appears in the getIndexes() output

Index Types

1. **Single Field Indexes:** Single field indexes collect, sort and store data from a single field in each document in a collection.

- By default, all collections have an index on the _id field. You can add additional indexes to speed up important queries and operations.
- You can create a single-field index on any field in a document, including:

✓ Top-level document fields

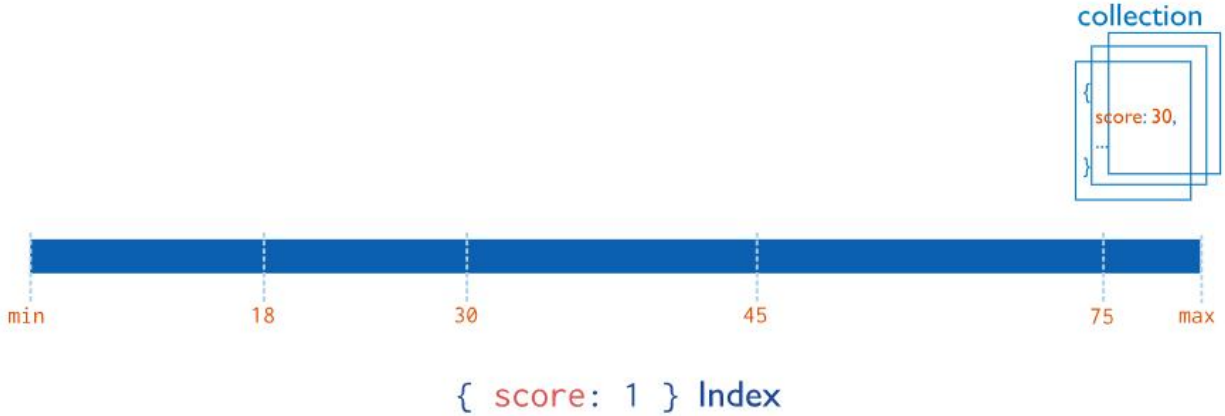
✓ Embedded documents

✓ Fields within embedded documents

- To create a single-field index, use the following prototype:

```
db.<collection>.createIndex( { <field>: <sortOrder> } )
```

- This image shows an ascending index on a single field, score:



In this example, each document in the collection that has a value for the score field is added to the index in ascending order.

Use Case:

If your application repeatedly runs queries on the same field, you can create an index on that field to improve performance. For example, your human resources department often needs to look up employees by employee ID. You can create an index on the employee ID field to improve the performance of that query.

Index Sort Order: For a single-field index, the sort order (ascending or descending) of the index key does not matter because MongoDB can traverse the index in either direction.

Example: Create a *students* collection that contains the following documents:

```
db.students.insertMany( [
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
] )
```

Create an Index on a Single Field: Consider a school administrator who frequently looks up students by their GPA. You can create an index on the gpa field to improve performance for these queries:

```
db.students.createIndex( { gpa: 1 } )
```

Results: The index supports queries that select on the field gpa, such as the following:

```
db.students.find( { gpa: 3.6 } )

db.students.find( { gpa: { $lt: 3.4 } } )
```

Create an Index on an Embedded Field: You can create indexes on fields within embedded documents. Indexes on embedded fields can fulfill queries that use dot notation.

```
db.students.insertMany( [
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
] )
```

- The location field is an embedded document that contains the embedded fields city and state. Create an index on the location.state field:

```
db.students.createIndex( { "location.state": 1 } )
```

Results: The index supports queries on the field location.state, such as the following:

```
db.students.find( { "location.state": "California" } )

db.students.find( { "location.city": "Albany", "location.state": "New York" } )
```

Create an Index on an Embedded Document

- You can create indexes on embedded documents as a whole.
- However, only queries that specify the **entire** embedded document use the index.
- Queries on a specific field within the document do not use the index.
- To utilize an index on an embedded document, your query must specify the entire embedded document. This can lead to unexpected behaviors if your schema model changes and you add or remove fields from your indexed document.
- When you query embedded documents, the order that you specify fields in the query matters. The embedded documents in your query and returned document must match exactly.

Example:

```
db.students.insertMany( [
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
] )
```

Create an index on the location fiel

```
db.students.createIndex( { location: 1 } )
```

Results: The following query uses the index on the location field:

```
db.students.find( { location: { city: "Sacramento", state: "California" } } )
```

The following queries *do not* use the index on the location field because they query on specific fields within the embedded document:

```
db.students.find( { "location.city": "Sacramento" } )
```

```
db.students.find( { "location.state": "New York" } )
```

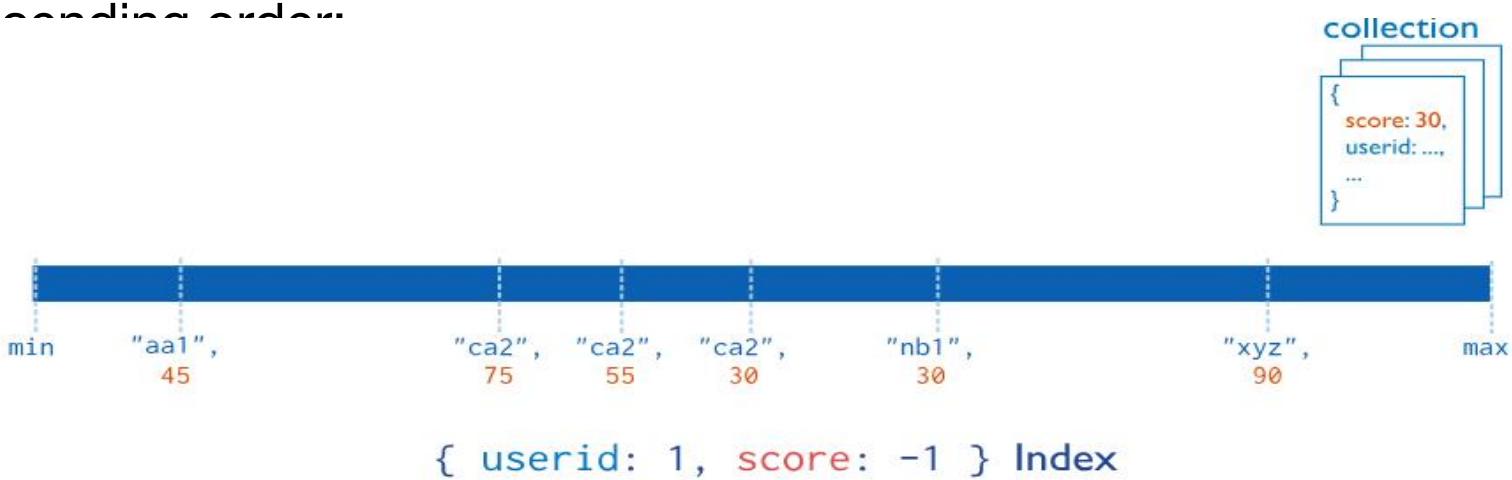
- In order for a dot notation query to use an index, you must create an index on the specific embedded field you are querying, not the entire embedded object.
- Following query returns no results because the embedded fields in the q

```
db.students.find( { location: { state: "California", city: "Sacramento" } } )
```

document:

2. Compound Indexes

- Compound indexes collect and sort data from two or more fields in each document in a collection.
- Data is grouped by the first field in the index and then by each subsequent field.
- For example, the following image shows a compound index where documents are first grouped by userid in ascending order (alphabetically). Then, the scores for each userid are sorted in descending order:



- Restriction: You can specify up to 32 fields in a single compound index.

- To create a compound index, use the following prototype:

```
db.<collection>.createIndex( {
   <field1>: <sortOrder>,
   <field2>: <sortOrder>,
   ...
   <fieldN>: <sortOrder>
} )
```

Use Case:

- If your application repeatedly runs a query that contains multiple fields, you can create a compound index to improve performance for that query. For example, a grocery store manager often needs to look up inventory items by name and quantity to determine which items are low stock. You can create a compound index on both the item and quantity fields to improve query performance.
- A compound index on commonly queried fields increases the chances of ***covering*** those queries. ***Covered queries*** are queries that can be satisfied entirely using an index, without examining any documents. This optimizes query performance.

Example: Create a students collection that contains these documents:

```
db.students.insertMany([
   {
      "name": "Alice",
      "gpa": 3.6,
      "location": { city: "Sacramento", state: "California" }
   },
   {
      "name": "Bob",
      "gpa": 3.2,
      "location": { city: "Albany", state: "New York" }
   }
])
```

The following operation creates a compound index containing the name and gpa fields:

```
db.students.createIndex( {
   name: 1,
   gpa: -1
} )
```

In this example:

- The index on name is ascending (1).
- The index on gpa is descending (-1).

Results: The created index supports queries that select on:

- Both name and gpa fields.
- Only the name field, because name is a prefix of the compound index.

For example, the index supports these queries:

```
db.students.find( { name: "Alice", gpa: 3.6 } )
```

```
db.students.find( { name: "Bob" } )
```

The index **does not** support queries on only the gpa field, because gpa is not part of the index prefix. For example, the index does not support th

```
db.students.find( { gpa: { $gt: 3.5 } } )
```

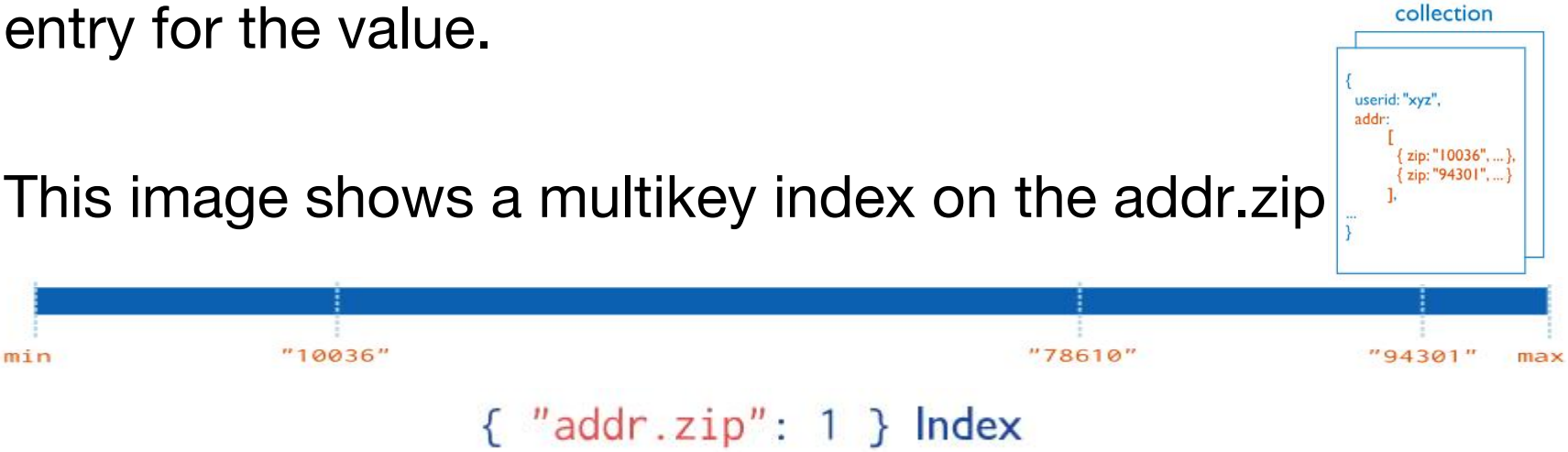
3. Multikey Indexes

- Multikey indexes collect and sort data from fields containing array values. Improve performance for queries on array fields.
- You do not need to explicitly specify the multikey type. When you create an index on a field that contains an array value, MongoDB automatically sets that index to be a multikey index.
- MongoDB can create multikey indexes over arrays that hold both scalar values (for example, strings and numbers) and embedded documents. If an array contains multiple insta

```
db.<collection>.createIndex( { <arrayField>: <sortOrder> } )
```

des one entry for the value.

- This image shows a multikey index on the addr.zip



Use Cases:

- If your application frequently queries a field that contains an array value, a multikey index improves performance for those queries.
- Indexing commonly queried fields increases the chances of ***covering*** those queries. Covered queries are queries that can be satisfied entirely using an index, without examining any documents. This optimizes query performance.
- For example, documents in a students collection contain a test_scores field: an array of test scores a student received throughout the semester. You regularly update a list of top students: students who have at least five test_scores greater than 90.
- You can create an index on the test_scores field to improve performance for this query. Because test_scores contains an array value, MongoDB stores the index as a multikey index.

Create an Index on an Array Field:

- You can create an index on a field containing an array value to improve performance for queries on that field. When you create an index on a field containing an array value, MongoDB stores that index as a multikey index.

```
db.<collection>.createIndex( { <field>: <sortOrder> } )
```

```
db.students.insertMany( [
   {
      "name": "Andre Robinson",
      "test_scores": [ 88, 97 ]
   },
   {
      "name": "Wei Zhang",
      "test_scores": [ 62, 73 ]
   },
   {
      "name": "Jacob Meyer",
      "test_scores": [ 92, 89 ]
   }
] )
```

You regularly run a query that returns students with at least one test_score greater than 90. You can create an index on the test_scores field to improve performance for this query.

```
db.students.createIndex( { test_scores: 1 } )
```

This operation creates an ascending multikey index on the test_scores field of the students collection.
Because test_scores contains an array value, MongoDB stores this index as a multikey index.

Results:

- The index contains a key for each individual value that appears in the test_scores field. The index is ascending, meaning the keys are stored in this order: [62, 73, 88, 89, 92, 97].
- The index supports queries that select on the test_scores field. For example, the following query returns documents where at least one element in the test_scores array is greater than 90:

```
db.students.find(
    {
        test_scores: { $elemMatch: { $gt: 90 } }
    }
)
```

Output

```
[
    {
        _id: ObjectId("632240a20646eaee87a56a80"),
        name: 'Andre Robinson',
        test_scores: [ 88, 97 ]
    },
    {
        _id: ObjectId("632240a20646eaee87a56a82"),
        name: 'Jacob Meyer',
        test_scores: [ 92, 89 ]
    }
]
```

Compound Multikey Indexes:

- In a compound multikey index, each indexed document can have ***at most* one** indexed field whose value is an array.
- Specifically: You cannot create a compound multikey index if more than one field in the index specification is an array. For example, consider a collection that contains this document:

```
{ _id: 1, scores_spring: [ 8, 6 ], scores_fall: [ 5, 9 ] }
```

- You can't create the compound multikey index { scores_spring: 1, scores_fall: 1 } because both fields in the index are arrays.
- If a compound multikey index already exists, you cannot insert a document that would violate this restriction. Consider a collection that contains the

```
{ _id: 1, scores_spring: [8, 6], scores_fall: 9 }
{ _id: 2, scores_spring: 6, scores_fall: [5, 7] }
```

- You can create a compound multikey index { scores_spring: 1, scores_fall: 1 } because for each document, only one field indexed by the compound multikey index is an array. No document contains array values for both scores_spring and scores_fall fields.
- However, after you create the compound multikey index, if you attempt to insert a document where both scores_spring and scores_fall fields are arrays, the insert fails.

Create an Index on an Embedded Field in an Array:

```
db.inventory.insertMany( [
   {
      "item": "t-shirt",
      "stock": [
         {
            "size": "small",
            "quantity": 8
         },
         {
            "size": "large",
            "quantity": 10
         },
      ]
   },
{
   "item": "sweater",
   "stock": [
      {
         "size": "small",
         "quantity": 4
      },
      {
         "size": "large",
         "quantity": 7
      },
   ]
},
   {
      "item": "vest",
      "stock": [
         {
            "size": "small",
            "quantity": 6
         },
         {
            "size": "large",
            "quantity": 1
         }
      ]
   }
] )
```

You need to order more inventory any time you have less than five of an item in stock. To find which items to reorder, you query for documents where an element in the stock array has a quantity less than 5. To improve performance for this query, you can create an index on the stock.quantity field.

Procedure: The following operation creates an ascending multikey index on the stock.quantity field of the inventory collection:

```
db.inventory.createIndex( { "stock.quantity": 1 } )
```

Because stock contains an array value, MongoDB stores this index as a multikey index.

Results:

- The index contains a key for each individual value that appears in the stock.quantity field. The index is ascending, meaning the keys are stored in this order: [1, 4, 6, 7, 8, 10].
- The index supports queries that select on the stock.quantity field. For example, the following query returns documents where at least one element in antity less than 5:

```
db.inventory.find(
    {
        "stock.quantity": { $lt: 5 }
    }
)
```

Output

```
[
  {
    _id: ObjectId("63449793b1fac2ee2e957ef3"),
    item: 'vest',
    stock: [ { size: 'small', quantity: 6 }, { size: 'large', quantity: 1 } ]
  },
  {
    _id: ObjectId("63449793b1fac2ee2e957ef2"),
    item: 'sweater',
    stock: [ { size: 'small', quantity: 4 }, { size: 'large', quantity: 7 } ]
  }
]
```